CHRONA

PROJECT REQUIREMENTS DOCUMENT

A voice-first Android planner that adapts to real life, connects related plans, and guides people without overwhelming them.

LOW PRIORITY Paper note	IMPORTANT Paper note	VERY IMPORTANT Paper note

Version: 1.0 - consolidated product requirements

Purpose: Source of truth for Claude architecture planning and OpenCode implementation prompts

Project context: University Research Experience (URE), two-credit project with a potential research and product roadmap

Target product: Android application; development must work from Ubuntu Linux and Windows

Status: Requirements baseline for review; technical architecture is to be planned after Phase 0 clarification

Core promise

Tell Chrona what you need to do. It creates a realistic plan, repairs it when the day changes, and brings the next task to your attention with a small, respectful interaction instead of a flood of notifications.

Prepared from the product decisions and ideas discussed in the project conversation, plus the supplied Universal Software Development Master Prompt.

Document map

This document is intentionally detailed. Claude should use it as the project requirements source, not as permission to skip its required clarification step. Sections 1-18 define the product and behavior; sections 19-24 define implementation guidance, testing, acceptance, research and remaining decisions.

Section	Content
1-4	Purpose, vision, problem, goals, non-goals and product principles
5-8	Users, primary journeys, app structure and visual direction
9-15	Functional requirements: voice, tasks, scheduling, recovery, Quest Cards, board/thread links and learning
16-18	Privacy, security, data concepts, integrations, accessibility and failure handling
19-21	Technical direction, development environment and phased delivery
22-23	Testing, evaluation metrics and acceptance criteria
24-26	URE/research pathway, risks and decisions Claude must validate
Appendices	Glossary, requirement index and authoritative technical references

Important workflow instruction

The attached Universal Software Development Master Prompt defines the planning workflow: Claude first reads the requirements, summarizes its understanding, asks only consequential clarification questions, then stops and waits. Claude must not immediately generate architecture or coding prompts. After clarification, it prepares the architecture pack and sequential prompts for one OpenCode coding agent working in one repository.

1. Project overview

1.1 Working name and concept

Chrona is the working name for a voice-first, adaptive daily planning application for Android. It combines natural task capture, realistic scheduling, schedule repair, a paper-and-thread task board, and brief game-inspired task cards. The product should feel like a calm personal planning environment rather than a generic AI chatbot or a conventional notification-heavy to-do list.

The name is provisional. Do not assume trademark availability, domain availability, or final branding has been checked.

1.2 One-sentence product definition

Chrona in one sentence

A voice-first Android planner that turns everyday requests into a realistic plan, automatically adjusts when life disrupts it, connects related tasks and events visually, and guides the user through small, well-timed task cards without constant interruptions.

1.3 Project context

The immediate context is a University Research Experience (URE) project worth two credits. The deliverable must be demonstrable and testable within a student project, but the design should leave a credible path toward a public app, a research paper, and later investigation of possible intellectual-property protection. These are aspirations, not guaranteed outcomes.

1.4 Intended development workflow

- Claude: understand requirements, ask clarification questions, define the architecture, repository structure, file responsibilities, shared contracts, module dependency map, testing plan, and sequential OpenCode prompts.
- OpenCode: implement the application in one repository using one coding agent/session, phase by phase; run tests; integrate modules; maintain progress and change logs; report actual verification results.
- The application itself is Android-first. Ubuntu and Windows are development hosts, not separate product targets in the initial scope.

2. Problem statement and opportunity

People often make plans using calendars and to-do lists, but real days change. A person wakes late, a class runs over, a task takes longer than expected, an unexpected event appears, or the planned workload was too large from the beginning. Once the original plan breaks, many people manually reorganize everything, abandon the rest of the day, or receive repeated reminders that do not solve the underlying problem.

Chrona should help with the gap between a plan on paper and a day as it actually unfolds. Its central purpose is not to maximize the number of tasks packed into every minute. It is to keep the user oriented, make realistic trade-offs, and help recover the important parts of the day.

2.1 Core user problems

- Adding tasks is often slower than remembering them; voice should make capture quick.
- Users may not know exact duration, priority, preferred time, or flexibility for every task.
- Ordinary schedules can become impossible after a disruption.
- Automatic schedulers can still feel like black boxes if they move tasks without explaining why.
- People already receive many messages and alerts; reminders can become noise and be dismissed reflexively.
- Tasks are not isolated: a project may prepare someone for an event, one task may block another, and a deadline may depend on earlier work.
- Users may worry about sharing personal schedules and behavioral history with a new app or external AI service.

2.2 Product hypothesis

> **Researchable hypothesis**
>
> A planner that combines realistic schedule recovery, explicit task/event relationships, and carefully limited context-aware task cards may help users complete important tasks with fewer unnecessary interruptions than a basic reminder list or manual rescheduling workflow. This must be tested; it is not yet a proven competitive advantage.

3. Goals, non-goals and product principles

3.1 Main goals

- Let users capture tasks quickly using natural speech, with typing as a full fallback.
- Create a feasible daily plan from incomplete but sufficient information, without a long setup questionnaire.

- Repair the plan when tasks are missed, delayed, added, or no longer feasible.
- Keep the user aware of the next useful action without flooding the notification shade.
- Provide a distinctive visual board where paper-like notes and red threads explain connections between tasks and events.
- Keep users in control: explain changes, allow correction, support undo, and avoid silently dropping tasks.
- Build privacy and cross-platform developer workflow into the design from the beginning.
- Produce reliable evidence from tests and a small user evaluation that can support the URE report and future research.

3.2 Non-goals for the first release

- Building a complete email client, note-taking suite, social network, or project-management platform.
- Importing all the user's Google, Microsoft, Apple, email, drive, and notes data by default.
- Always-listening microphone or covert activity monitoring.
- Claiming to be the first app to use voice, floating cards, adaptive scheduling, or gamification.
- Guaranteeing a perfect schedule or claiming that every task can be completed.
- Using guilt, shame, streak loss, punitive scoring, or manipulative gamification to force compliance.
- Implementing many speculative AI agents or a complex microservice system without a demonstrated need.

3.3 Product principles

Principle	Practical interpretation
Low friction	A new user can add a first task immediately. Ask only questions that materially change the plan.
Realistic over optimistic	Respect actual available time, deadlines, fixed events, estimates and recovery buffers.
Quiet by default	Use a persistent but unobtrusive task surface and a small number of useful interruptions.
Explainable	Every meaningful schedule change can be explained in plain language.
User control	The user can accept, edit, dismiss, postpone, undo, disable overlays and disconnect integrations.
Privacy by design	Collect the minimum information needed, request permissions at the point of need and avoid broad account access.
One coherent product	All phases are integrated into one application and one repository using canonical shared contracts.
Evidence, not claims	Treat differentiation and research conclusions as hypotheses until supported by competitor review and user testing.

4. Target users and user stories

4.1 Initial audience

The first audience is students and busy individuals managing study, assignments, exercise, personal goals, appointments and events. The product should also be understandable to a non-technical user; the interface must not assume that people understand AI, scheduling algorithms, or project-management terminology.

4.2 Key user stories

ID	User story	Expected outcome
US-01	As a user, I want to say “Add GATE study for tomorrow” so I can capture it immediately.	Chrona creates a task and asks only if a missing detail is essential.
US-02	As a user, I want Chrona to plan my day around fixed commitments.	No scheduled task overlaps a fixed commitment.
US-03	As a user, I woke up late and want to recover my day quickly.	Chrona offers a revised feasible plan and explains affected tasks.
US-04	As a user, I want a small task reminder over my current app.	A temporary Quest Card appears if allowed and appropriate; it can be dismissed.
US-05	As a user, I do not want repeated reminders.	Chrona respects my cap, quiet hours and chosen interruption mode.
US-06	As a user, I want to see that a project helps me prepare for an event.	A red thread connects related notes and explains the relationship.
US-07	As a user, I want to correct an AI misunderstanding.	I can edit task details before or after saving; errors do not corrupt the plan.
US-08	As a user, I want to use Chrona without connecting my accounts.	Core manual/voice task management works without external account integrations.
US-09	As a user, I want to delete my data.	The app exposes a clear deletion and export path appropriate to the chosen storage design.
US-10	As a user, I want to understand why my day changed.	Chrona records a simple schedule change history and a human-readable reason.

5. Product experience and main navigation

5.1 Main areas

Area	Purpose	Design notes
Today	Immediate daily guidance: next task, current plan, completion and upcoming tasks.	Simple, calm and action-focused. Always provide a clear “what next?” answer.
Schedule	Time-based day/week view and task movement.	Make conflicts and fixed commitments clear; avoid forcing users into a dense project-management calendar.
Board	Paper-like corkboard view with colored notes and red threads.	Supports project thinking, connections and future events; filters reduce clutter.
History	Shows completed, postponed, skipped and automatically moved tasks.	Explain what changed and allow appropriate undo/recovery.
Settings	Voice, notifications, overlay, privacy, data, integrations, accessibility and preferences.	Plain language. Show why permissions are needed before asking for them.

5.2 Key actions always close at hand

The app should make the following actions easy to find: speak/add a task, type/add a task, see the next task, mark done, postpone, change schedule, see the reason for a change, and open the full app. Avoid a home screen filled with statistics and complex controls.

5.3 Two complementary views

- Today view is optimized for execution: “What should I do next?”
- Board view is optimized for context: “How are my tasks, projects, dependencies and future events connected?”

The same underlying task records must power both views. A task should not become a separate copy when viewed on the board versus the schedule.

6. Visual language: paper notes, pins and red thread

6.1 Overall art direction

Use the provided reference image as inspiration, not as a template to copy literally. The desired feeling is a real, tactile planning/investigation board: warm cork or paper texture, pinned note cards, slight shadows, subtle rotations, metal pin details and thin red string connecting related items. Avoid a cartoonish or overly literal 3D look; keep the interface polished, readable, modern and lightweight.

6.2 Priority colors

Priority concept	Proposed paper treatment	Meaning
Low / side task	Soft yellow note	Useful but deferrable; should not displace urgent or important work.
Important / high priority	Soft blue note	Needs attention soon or has meaningful importance.
Very important / critical	Soft green note with a clear priority label or marker	Protect carefully, especially when linked to a deadline or commitment.
Completed / archived	Desaturated paper or folded-corner treatment	Visually lower emphasis; remain accessible in history/filters.

The exact mapping of priority labels and colors remains a product detail Claude may refine during design, but preserve the established direction: yellow for lower-priority/side tasks, blue for important tasks, green for very important tasks. Do not rely on color alone; include labels/icons for accessibility.

6.3 Note contents

- Task title (short and legible).
- Planned time or due date when relevant.
- Small priority/status symbol.
- Optional duration or project/category mark.
- A clear selected/expanded state and accessible tap target.
- Subtle pin/shadow/rotation treatment, never to the point that text is difficult to read.

6.4 Red-string behavior

Red threads represent a meaningful relationship between two notes, even when the notes have different priorities or appear on different dates. A thread must not imply that two tasks have the same priority. When a user selects a thread, Chrona should reveal the relationship type and an explanation.

Relationship type	Example	Scheduling implication
Supports / helps prepare for	Building a project helps prepare for a future showcase.	Both tasks relate, but the showcase does not necessarily block work on the project.
Depends on / blocked by	Submit an application after completing a required document.	The prerequisite should be completed before the dependent task.
Part of project	Research, implementation and testing all belong to one project.	Group/filter together; dependencies are separate from grouping.
Same event or outcome	Register for an event and later attend it.	Keep the relationship visible and preserve event timing.
Related / reference	Read a paper that informs a later assignment.	Display the connection without automatically enforcing order.

The app must distinguish a dependency ("B cannot happen before A") from a helpful relationship ("A may help B"). This distinction is important to schedule logic. Connections may be created by the user or suggested by Chrona, but automatically suggested links should be visible and easy to accept, edit or reject. Do not silently invent consequential dependencies.

6.5 Board usability

- Allow zoom, pan, and filtering by project, date, status and priority.
- Allow the user to focus on a project or a subset of related tasks so the board does not become a tangled web.
- Keep string paths behind or around cards where possible; lines must not obscure note text or controls.
- Provide a list/accessible alternative to the visual board.
- When a task moves to another date, preserve meaningful relationships and make the move visible.
- Use lightweight animation and avoid expensive constant rendering that drains battery.

7. Voice-first task capture

7.1 Voice is primary, typing is fully supported

Voice should be one of the most prominent interactions in Chrona. A user can tap a microphone control in the app or use an optional floating voice entry control to say a task. Typing is essential when speaking is inconvenient, unavailable, noisy or inaccessible.

7.2 Expected natural-language examples

User says/types	Expected interpretation
"I need to study for GATE for two hours tomorrow."	Create a study task, approximately 120 minutes, date tomorrow; ask for a time only if needed to place it responsibly.
"Finish my assignment before Friday."	Create task with Friday deadline; use known preferences/availability to propose time.
"Move my workout to later; I woke up late."	Treat as a schedule recovery request, not just a new task; preserve fixed commitments.
"My class ran late; move everything that can be moved."	Update actual availability, identify affected tasks and propose/apply a recovery plan according to user's automation preference.
"What should I do next?"	Return the current next task and its reason, using a compact spoken/text response.
"I finished the assignment."	Mark the matching task complete after identifying it with sufficient

User says/types	Expected interpretation
	confidence.
“Connect this project to my showcase event.”	Create or propose a relationship with a meaningful type; ask only if the relation is ambiguous.
“I need a break.”	Offer a short break and adjust flexible work if required; do not treat this as task failure.

7.3 AI task understanding behavior

- Convert speech/text into structured task intent: title, date/deadline, duration estimate, priority clues, fixed/flexible status, split preference and relationship intent where present.
- Return structured, validated data rather than relying on unconstrained prose as the only output.
- Use confidence and ambiguity handling. Ask one concise question only when a missing detail could change a consequential outcome.
- Use safe defaults for low-risk missing fields and show the assumption where helpful.
- Never make the user answer a long questionnaire for every task.
- Allow easy correction of recognition and interpretation mistakes.
- AI interprets language; the scheduler is responsible for checking constraints and making schedule placement decisions.
- If remote AI is used, send only the minimum task text and context required; do not send the entire calendar or history by default.

7.4 Microphone privacy and platform behavior

The default interaction is explicit tap-to-speak. Chrona must not continuously record ambient audio or imply that it listens when it does not. Explain microphone permission at the point the user starts voice capture. Provide clear recording/listening state, cancel, retry and typing alternatives. Any future wake-word or always-listening mode would require a separate privacy, battery, platform and consent review.

8. Minimal onboarding and preference setup

8.1 First-use objective

A user should be able to create the first task and see useful value quickly. Aim for approximately one minute to reach the first task as a design target, not a guaranteed metric. Do not block first use on account creation, calendar connection or a long preference survey unless the eventual architecture requires a specific reason that Claude must explain.

8.2 Minimum onboarding choices

- Optional preferred start/end of day or select “Flexible”.
- Reminder style: Quiet, Balanced or More Guidance; explain each in ordinary language.
- A clear option to skip preferences and adjust them later.
- Request microphone permission when the user first taps voice capture, not as an unexplained first-launch demand.
- Request notification permission when the value of reminders has been demonstrated and explain the benefit first.
- Offer overlay permission only when the user enables Quest Cards across other apps; explain that it is optional and may not work on every screen/device.
- Let the user add the first task by voice or text immediately.

8.3 Proposed starting defaults

Setting	Proposed default (subject to validation)	Reason
Reminder style	Quiet / low-interruption	Earn trust before increasing interruption frequency.
Non-urgent Quest Card cap	Small daily cap; test 2 per day as an initial hypothesis	Avoid becoming another source of noise.
Quiet hours	Ask optionally or default to device-aware sensible suggestion; make editable	Avoid sleep and user-declared focus periods.
Voice mode	Tap-to-speak only	Simple and transparent privacy posture.
External account connections	None by default	The core product should work without integration setup.
Task duration missing	Use a visible reasonable estimate based on task category or a conservative default; let user adjust	Avoid interrupting the user over every missing field.

These are suggested starting assumptions for Claude to validate, not immutable product requirements. Any change should preserve the low-friction, quiet-by-default direction.

9. Tasks, events and schedule records

9.1 Task information

At minimum, a task should be able to store the fields below. The final technical schema should be documented by Claude in the shared contract and data-model files.

Field/concept	Purpose	Notes
ID	Stable task identity	Must remain stable when the task moves in time or appears in different views.
Title and optional details	Human-readable task	Validate size; show a short title on the paper note.
Priority	Low, important, very important/critical	Use both visual treatment and explicit data; never infer solely from color.
Estimated duration	Expected amount of work	Can be learned over time; keep a manual correction option.
Deadline / due date	Latest acceptable completion time	Different from preferred schedule time.
Earliest start / preferred window	When work can or should occur	Preference, not necessarily a hard rule.
Fixed / flexible	Whether time can move	Fixed commitments must not be moved by automatic recovery.
Splittable and minimum session	Whether a long task can be split safely	Do not split every task by default; preserve minimum useful session length.
Status	Inbox, planned, in progress, completed, postponed, skipped, archived	Transitions must be explicit and testable.
Origin	Voice, typed input, imported calendar, manual edit	Use for debugging/traceability without storing unnecessary raw audio.
Project/category and relations	Groups and linked tasks/events	A task may link to multiple other records.

Field/concept	Purpose	Notes
Actual duration/outcome	Supports better estimates	Optional and privacy-conscious; avoid judging users.

9.2 Events and fixed commitments

Events are time-bound commitments such as classes, meetings, appointments or an event the user plans to attend. Keep their start/end time and timezone explicit. Imported external calendar events should be handled according to user-granted access; do not assume Chrona can freely read or edit every calendar.

9.3 Task status semantics

Action	Required interpretation
Dismiss card	Only closes the card. It must not mark the task completed or silently delete it.
Done	Marks the task completed with timestamp and updates the schedule/history.
Later	Requests a new time or a schedule repair; do not repeatedly snooze without limits.
Skip	Records that the user chose not to do the task; should preserve the record and allow rescheduling/replanning.
Delete	An explicit destructive action; distinguish from dismiss/skip and offer confirmation/undo where appropriate.
Undo	Restores the affected schedule/task state if still safe; handle stale or conflicting undo operations clearly.

10. Realistic scheduling engine

10.1 Key design decision

> **Use AI for understanding; use explicit scheduling logic for feasibility**
>
> AI can interpret what a person means. A scheduling engine must decide where work fits and then validate that the resulting plan respects fixed events, deadlines, task rules and available time. Do not treat a fluent AI-generated timetable as proof that the schedule is valid.

10.2 Scheduling inputs

- Usable time windows and user-defined working hours.
- Existing fixed events and commitments.
- Task duration estimates, deadlines, earliest start and preferred time windows.
- Priority/importance and what happens if a task is late.
- Task flexibility, split eligibility and minimum session length.
- Dependencies and relationship types, with a distinction between hard dependencies and merely helpful links.

- Current progress, skipped work and actual availability after a disruption.
- User settings such as break preferences, focus/quiet periods and recovery mode.

10.3 What makes a schedule realistic?

- No impossible time overlap: a user cannot be scheduled for two tasks at the same time unless one is explicitly allowed as a background/parallel activity.
- Fixed commitments are preserved unless the user explicitly changes them.
- Total scheduled duration fits into available windows, taking travel/buffer/break time into account when applicable.
- Deadlines and dependency order are checked, not merely task priority.
- Estimated duration is a starting point and should adapt based on actual completion history only when the user allows that learning.
- Tasks that cannot fit are clearly labelled as unresolved, at risk or needing a trade-off; they must not disappear silently.
- If a long task is split, every segment meets the task's minimum useful length and the combined duration is adequate.
- The engine records why a task was moved or left unscheduled.

10.4 Scheduling algorithm expectations

Claude should choose an appropriate deterministic strategy after architecture planning. The implementation should support evaluating candidate plans against explicit constraints and a scoring function, rather than relying on scattered priority if-statements. A simple MVP can use a deterministic slot-placement algorithm; later versions may use constraint optimization if tests show the need. Do not introduce complex optimization dependencies without justification.

10.5 Candidate schedule scoring

A candidate schedule may be evaluated using weighted considerations such as: hard constraint violations (which should be disallowed), missed deadline risk, task importance, prerequisite order, user preferences, total amount of movement, task fragmentation, unused gaps, and recovery burden. Weights should be documented and unit-tested. Exact weights remain an architecture/product decision that should be tuned against test cases rather than invented as a universal truth.

10.6 Overloaded-day behavior

When demand exceeds available time, Chrona must be honest. It should show which work fits, which work is at risk, and alternatives such as moving low-priority work, splitting a task, postponing it, shortening a session if useful, or asking the user to choose between consequential trade-offs. It must never claim that every task fits when the sum of required work exceeds the available time.

11. Schedule repair and recovery

11.1 Trigger examples

- User wakes later than planned or starts the day late.
- A fixed class/meeting runs longer than expected.
- A task takes more time than its estimate.
- A task is skipped, postponed or not started.
- An unexpected event is added.
- The user adds a new task or changes its deadline/priority.
- The user says they are tired, overwhelmed or need a break.
- A task becomes irrelevant or its dependency changes.

11.2 Recovery flow

1. Detect what changed and identify the affected part of the schedule.
2. Update actual availability and identify fixed commitments that must stay put.

3. Check dependencies, deadlines, duration estimates, priority and flexibility.
4. Generate one sensible recommended plan; offer alternatives only if there is a meaningful trade-off.
5. Validate the plan before applying it. Do not commit a plan that fails hard constraints.
6. Apply according to the user's automation preference: suggest first, auto-move flexible tasks, or use user-approved recovery rules.
7. Explain the important changes in concise language and offer undo.
8. If no feasible schedule exists, state what cannot fit and ask for the smallest necessary decision.

11.3 Recovery modes (proposed experience)

Mode	Use case	Expected behavior
Rescue My Day	Late start or significant disruption.	Preserve fixed commitments and protect essential outcomes; move flexible tasks with minimal friction.
Reduce My Load	Workload feels too large or user is overwhelmed.	Make the plan more realistic; propose moving optional tasks or shortening/splitting work where useful.
Get Me Back on Track	A task has been postponed repeatedly.	Try a smaller first step, a better time, or a session split; avoid labelling the user as lazy or blaming them.
Normal planning	No major disruption.	Schedule tasks normally while leaving sensible buffer and honoring user settings.

These mode labels are a product idea, not a required scientific classification. Keep the underlying behavior understandable and let users access the normal workflow even if they do not want modes.

11.4 Recovery history and explanation

Record schedule revisions as compact events: trigger, tasks affected, old and new times, status of the plan, reason, and whether the user accepted/rejected/undid the change. The UI should explain decisions without exposing internal model reasoning or overwhelming the user. Example: "You started two hours late. I moved the optional reading to tomorrow and kept your assignment before its deadline."

12. Quest Cards: quiet, game-inspired reminders

12.1 Purpose

Quest Cards are small, temporary, paper-inspired or game-achievement-inspired cards that bring the next task to attention and then disappear. They are not standard persistent notification banners for every task. The user should be able to continue using the current app without opening Chrona.

12.2 Trigger policy

Before showing a card, Chrona's reminder manager should consider: task importance and due time, current reminder count, recent dismissal/deferral, quiet hours, active focus settings, whether the task is still the next useful action, whether another card was recently shown, and whether the current device/app state makes an overlay inappropriate. The exact signals available to Android are limited; do not promise awareness of other apps' private state or monitor other apps' content.

12.3 Card behavior

Behavior	Requirement
Appearance	Small rectangular/rounded card, tactile note styling, readable contrast, restrained motion; avoid covering essential controls.
Content	Task title, short context, approximate duration or due clue, and at most one primary action.
Automatic disappearance	Disappears after a short configurable display interval; the task remains in Chrona. Suggested initial animation timing should be tuned in usability testing.
Tap	Opens a compact task panel, not the full app by default.
Start	Marks a task as in progress and can start a timer/session if implemented.
Done	Marks complete; may show a small optional success response.
Later	Starts a controlled postpone/recovery flow and respects snooze limits.
Dismiss / close	Closes card only. It does not mark completion.
Reopen	From the main app, widget or a permitted overlay control; no need to search for a vanished notification.
Accessibility	Supports large text, screen readers, reduced motion preferences and clear controls.

12.4 Reminder caps and quiet behavior

- Allow Quiet, Balanced and More Guidance modes; users can set exact caps and quiet hours in settings.
- Use a small default cap for non-urgent cards; test two non-urgent cards per day as a research hypothesis, not as a fixed universal rule.
- Batch several upcoming low-importance tasks into one useful summary where it makes sense.
- Avoid repeated re-notification after the user dismisses a card. A reminder must not repeat until the configured policy or a materially changed condition allows it.
- Keep genuinely important deadline-risk alerts rare, explainable and configurable; do not use “critical” as a loophole to bypass every cap.
- Make it possible to disable overlays while preserving the in-app schedule and any Android notification fallback the user allows.
- Never count a dismissed card as completed or assume the user has seen/read it.

12.5 Overlay permission and fallback

A custom card displayed over other apps will require the appropriate Android overlay permission and cannot be guaranteed to appear over every screen or on every device. It must be optional, initiated by a user choice, and accompanied by a clear explanation of why permission is needed. Provide a normal Android notification or in-app/widget fallback if the overlay is disabled or unavailable. Respect Android system restrictions and test real devices, not only an emulator.

12.6 Gamification principles

A completed task may trigger a subtle “Quest complete” response, achievement title or small visual reward. Gamification should reinforce completion without creating shame, compulsory streaks, manipulative urgency or punishment for missing tasks. Allow animations/rewards to be reduced or disabled. The paper-and-thread visual theme is the foundation; game metaphors are optional supporting behavior.

13. Board relationships and thread logic

13.1 Relationship data and semantics

Each relationship should have stable source and target IDs, a relationship type, optional explanation, creation source (user or Chrona suggestion), confirmation state, and timestamps as appropriate. Claude should define exact types and constraints in canonical shared contracts.

13.2 Required behaviors

- Users can connect tasks and future events regardless of priority or date.
- The relationship remains attached to stable records as schedule times change.
- Deleting or archiving a task must handle incoming/outgoing links predictably and avoid orphaned connections.
- Dependency relations may constrain scheduling; supportive/related relations normally should not constrain time unless explicitly configured.
- Suggested relationships must be reviewable, editable and rejectable.
- The board must explain the relationship when tapped; a thread should never be decorative with no meaning.
- Provide list/search alternatives to locate linked tasks without navigating a dense board.

13.3 Example scenario

Project -> future event

A user is building a project and has registered for a showcase event next month. The project note and the showcase event note appear on different dates. A red thread links them with the explanation “This project is intended for presentation at this event.” Completing the project may help prepare for the event, but the thread alone does not mean the event must move or that the project is a strict scheduling prerequisite.

14. Personalization and learning over time

14.1 What the app may learn

- Actual duration compared with estimated duration, if the user chooses to record it.
- Common preferred working windows and task categories.
- Which reminder modes/caps the user keeps enabled and which actions they take.
- Repeated postponement patterns, used to offer practical assistance rather than judge the person.
- Which kinds of schedule changes the user accepts, edits, rejects or undoes.
- Typical recovery outcomes, including how much manual correction the plan needed.

14.2 Learning safeguards

- Do not describe users as lazy, undisciplined or failures.
- Do not infer sensitive personal traits or health conditions from task behavior.
- Expose a way to correct or reset learned preferences.
- Let users disable personalization and, where practical, inspect the preference data used.
- Do not send an entire behavioral history to an AI model by default.
- Avoid training a global model on personal data unless there is a separate, clear, consented and legally reviewed plan.

14.3 Delivery stage

Advanced personalization is not required to prove the first end-to-end product loop. Implement a modest history model first and add learning only after the basic scheduler and recovery behavior can be evaluated objectively.

15. Main screens and interaction requirements

Screen / surface	Required contents and actions
First launch	Short explanation of value; skip-friendly preferences; no forced integrations; first task by voice or text.
Today home	Next task, brief daily status, remaining tasks, schedule preview, prominent voice button, quick completion/postpone/change actions.
Schedule	Day view first; optional week view; fixed commitments; task blocks; conflicts; task movement; clear recovery entry point.
Paper Board	Colored paper notes, pins, red thread connections, zoom/pan/filter/search, project focus, accessible list view.
Quest Card overlay	Temporary card and compact task panel with Start, Done, Later, Dismiss and optional task details.
Recovery proposal	Short summary of what changed, why, affected tasks and key unresolved trade-off; Accept/Apply, Edit, Alternative, Undo as appropriate.
Task editor	Title, duration, priority, deadline, earliest/preferred time, fixed/flexible, split setting, links and notes; fields can be optional.
History	Completed/postponed/skipped tasks, plan revisions, reasons, undo where safe.
Settings	Reminder style and cap, quiet hours, overlay toggle, reduced motion, voice settings, privacy/export/delete, integrations and permissions.
Privacy centre	Plain-language data inventory, active integrations/permissions, disconnect controls, export/delete options and AI data handling explanation.
Error/retry states	Plain-language explanation, retry or manual fallback; never expose secrets or stack traces to end users.

16. Non-functional requirements

ID	Requirement	Priority / acceptance intent
NFR-UX-001	Core interactions should be understandable without technical knowledge. Prefer simple language, low cognitive load, clear task status and a small number of actions.	High; usability review with first-time users.
NFR-UX-002	First useful task should be possible without completing a long onboarding flow.	High; target about one minute, validate in test.
NFR-PERF-001	Common local actions (open Today, mark done, view a task) should feel immediate on supported test devices.	High; set measurable timing targets during architecture/QA.
NFR-PERF-002	Board rendering and animations should stay responsive with a realistic set of tasks and	High; load/performance test with at least 100 tasks and 200 relationships as a starting fixture.

ID	Requirement	Priority / acceptance intent
	connections.	
NFR-REL-001	Task creation and schedule changes must persist safely; partial failures must not silently lose or duplicate actions.	High; persistence and retry tests.
NFR-REL-002	The app must handle unavailable network/AI service using graceful fallback; core viewing/editing of stored tasks should not depend on a live LLM.	High; offline/failure tests.
NFR-PRIV-001	Collect only the data needed for the feature, with clear purpose, retention and deletion rules.	Critical; privacy review.
NFR-SEC-001	Secrets and provider API keys must not be embedded in a distributable Android client.	Critical; static/security audit.
NFR-SEC-002	User records must be isolated; remote authorization must be enforced server-side, not only by client UI.	Critical if cloud storage is used; negative authorization tests.
NFR-ACC-001	Color is not the only indicator of priority/status; support scalable text, accessible controls and reduced motion.	High; accessibility review.
NFR-COMP-001	The repository must build from Ubuntu and Windows using documented, pinned project tooling and wrapper scripts.	Critical; CI/local verification on both operating systems.
NFR-MAINT-001	Architecture, shared contracts, dependencies and important state transitions must be documented and tested.	High; contract and architecture audits.

17. Privacy, security and user trust

17.1 Privacy position

> **The privacy promise must match the implementation**
>
> Do not tell users that their data is “already public” or that privacy does not matter because large platforms possess data. Private calendar entries and notes are not automatically public. Chrona must earn trust with limited collection, clear permissions, secure handling and meaningful user control.

17.2 Data-minimization requirements

- Core scheduling should work with tasks entered directly in Chrona; external integrations are optional.
- Do not request all files, all email, all notes or broad account permissions to solve task scheduling.
- Prefer narrow, read-only integration scopes when read access is sufficient. Request write access only for an explicit feature that needs it.
- Explain what data goes to an external AI service and for what purpose.
- Send only the minimum text/context needed to interpret a task; do not include a full calendar by default.
- Avoid retaining raw voice recordings unless a feature requires it and the user is informed. Store structured task data rather than unnecessary audio where possible.
- Provide account/data export and delete paths consistent with the selected local/cloud data architecture.

- Keep diagnostic logs free from task content, OAuth tokens, API keys, raw audio and other sensitive data unless specifically redacted and justified.

17.3 Security requirements

- Use Android secure storage facilities for local secrets where applicable; do not store plaintext access tokens in ordinary task records.
- Use encrypted transport for network communication; define appropriate at-rest protection for local and server data.
- Never place private provider API keys in app source, resources or build artifacts. Any key required for a third-party AI provider should be protected behind a server-side service if that provider's design requires a secret key.
- Validate all user and AI-produced structured input before database writes or schedule actions.
- Use least-privilege access rules, user-scoped queries and server-side access controls if remote data storage is added.
- Use OAuth/official APIs for integrations; never ask users to provide third-party account passwords to Chrona.
- Support revoking a connected account and handle expired/revoked tokens gracefully.
- Provide transparent error states without leaking sensitive context or implementation details.
- Document data fields, purpose, retention, flow to each external service and deletion behavior before public release.

17.4 Privacy language for product screens

Suggested language to adapt after the technical design is known: "Your plans belong to you. Chrona uses the information needed to organise your tasks. You can start without connecting external accounts. If you connect a service or use an external AI feature, Chrona will explain what information is shared and why. You can manage connections and use the available data export and deletion controls." Do not publish this text until the implementation actually supports its claims.

17.5 Privacy acceptance criteria

- Every permission request explains a user-visible feature benefit before the system prompt.
- Denial of optional permissions does not break unrelated core features.
- No unexpected audio capture is possible in normal tap-to-speak mode.
- External integration access is disclosed, optional and revocable.
- Sensitive data and secret scans are part of final verification.
- Delete/export flows are tested against local and remote records, including recovery/history where applicable.

18. Android platform, integrations and developer environment

18.1 Target platform

The initial product target is Android. The app does not need a separate Windows or Ubuntu desktop UI for the URE MVP. The Windows PC and Ubuntu laptop are both development machines and must be able to open, build, test and continue the same codebase without machine-specific redesign.

18.2 Recommended technical direction (not an immutable architecture)

Area	Recommended starting direction	Architecture owner must validate
Android UI	Native Android app; Kotlin with Jetpack Compose is a strong default for a modern Android UI.	Current stable compatible versions, minimum supported Android version, accessibility and animation needs.
Build	Gradle wrapper and one documented JDK/Android SDK baseline.	Exact JDK, Gradle, Android Gradle Plugin, SDK and Kotlin versions after compatibility check.
Local data	Local-first task/schedule persistence (e.g.,	Data migration, backup strategy, local encryption

Area	Recommended starting direction	Architecture owner must validate
	Room/SQLite) is a reasonable default for an MVP.	need and offline behavior.
AI task parsing	An interface that can call an approved model/API, with local/mock parser for deterministic tests.	Provider, cost, privacy policy, prompt/schema validation, rate limits and fallback.
Scheduling engine	Deterministic domain module independent from UI and AI provider.	Algorithm, constraints, objective/scoring and test fixtures.
Background work	Android-supported scheduling/work APIs selected according to actual need.	Battery, exact timing, background restrictions and device behavior.
Overlay / Quest Card	Optional Android overlay with system permission; fallback notification/in-app card.	OS restrictions, lifecycle, touch accessibility, app store/policy implications, OEM-specific behavior.
Calendar integration	Optional later integration via official user-authorized APIs.	Provider, exact scopes, sync direction, conflict behavior and deletion semantics.
Remote sync	Not mandatory for first local demonstrator unless user confirms need.	Account model, backend, authorization, storage region, costs and backup.

18.3 Android-specific constraints

Android controls when an app can display overlays, start background work, access the microphone and post notifications. The app must not promise that a custom card can appear above every application, that reminders will fire at an exact second on every device, or that voice capture can start silently from the background. Request overlay and notification permissions transparently. Voice capture should be user-initiated. Validate restrictions against the Android versions chosen by the architecture.

As of the official Android documentation consulted for this draft, Android 13 (API level 33) introduced a runtime notification permission; notification channels allow users to adjust notification behavior; and Android background foreground-service/microphone behavior is subject to version-specific restrictions. Claude/OpenCode must verify the current official guidance at implementation time rather than relying on this static summary.

18.4 Ubuntu/Windows parity requirements

- Keep the canonical code in one Git repository. Use the same Gradle wrapper and compatible Android SDK/JDK versions on both hosts.
- Document installation and build steps for Ubuntu and Windows separately, but make the actual code and build output the same.
- Avoid absolute home-directory paths, drive-letter assumptions, shell-only build steps, case-insensitive filename assumptions and manual untracked local edits.
- Use repository scripts that have Windows and Unix equivalents where needed (for example, Gradle wrapper scripts); document equivalent commands.
- Pin dependencies and commit lockfiles/wrapper configuration where appropriate.
- Use .editorconfig and consistent line endings; avoid filenames that differ only by case.
- Keep secrets in local ignored environment files or a secure secret mechanism; provide a clearly named example configuration with no real secrets.
- Test emulator and/or physical-device execution from both development hosts where practical. Physical overlay and microphone behavior must be validated on a real Android device.
- Record toolchain versions and exact build/test commands in the README and final verification report.

18.5 Optional integrations and scope boundary

Do not start with “connect everything”: Google Calendar, Google Keep, Microsoft, Apple, email, cloud drives and all other accounts are not MVP requirements. Prioritize integrations only after the core behavior is validated. The first integration

candidate is a calendar because fixed events affect scheduling; access must be explicit and permission-limited. Notes/files/email access requires a separate feature rationale, authorization review and user consent.

19. Conceptual architecture and key data flows

The following is a product-level model for Claude to turn into a technical architecture. It is not a substitute for the architecture, directory tree, file map, or shared contracts required by the Universal OpenCode prompt.

Layer / component	Responsibility	Must not do
User interface	Voice/text entry, Today, Schedule, Board, Quest Card, settings and feedback.	Must not embed scheduling rules across UI callbacks.
Input / intent parser	Convert speech/text to structured task commands and confidence.	Must not be the final authority on schedule feasibility.
Validation/domain layer	Validate task fields, permissions, status transitions and relationship types.	Must not trust raw LLM output or client-supplied data blindly.
Task and relationship store	Persist stable task/event IDs, relations, history and user preferences.	Must not create disconnected duplicate records for every view.
Scheduling/recovery engine	Generate candidate schedules, check constraints, choose a plan and explain changes.	Must not silently drop work or violate hard constraints.
Reminder decision manager	Select which card/notification is appropriate under preferences and policy.	Must not treat every task due time as permission to interrupt.
Android presentation adapters	Display permitted overlay, notification, widget or in-app states.	Must not assume overlay/microphone/notifications are always permitted.
AI provider adapter	Submit limited input to approved service and parse structured result.	Must not hold provider secrets in distributable client code.
Audit/history and evaluation	Record schedule revisions, outcomes and consented research metrics.	Must not collect unnecessary personal content or expose internal sensitive logs.

19.1 Canonical data concepts

Entity	Key contents / relationships
User preferences	Timezone, preferred hours, default durations, reminder mode/cap, quiet hours, overlay setting, reduced motion, learning consent.
Task	Stable ID, title, details, priority, estimate, deadline, preferred/earliest window, fixed/flexible, split settings, state, category/project.
Event / commitment	Stable ID, title, start/end, timezone, fixed/movable, origin/integration metadata and sync status where applicable.
Schedule block	Task/event ID, start/end, status, source of placement, revision and whether the slot is proposed or accepted.
Task relationship	Source ID, target ID, relation type, explanation, source (user/suggested), confirmation state.
Schedule revision	Trigger, old/new schedule changes, explanation, feasibility result, accept/reject/undo state, created time.
Interaction history	Minimal action event such as card shown/dismissed, task completed, postponed, estimate corrected; configurable and privacy-conscious.
Integration connection	Provider, granted scopes, connection state, token metadata stored

Entity	Key contents / relationships
	securely, last sync/result; no raw password.

19.2 Shared contracts and repository records

Claude should create the canonical documents mandated by the Universal OpenCode prompt, adapted to this Android project. At minimum: MASTER_REQUIREMENTS.md, REQUIREMENT_TRACEABILITY.md, MASTER_ARCHITECTURE.md, PROJECT_DIRECTORY_TREE.md, FILE_RESPONSIBILITY_AND_PHASE_MAP.md, SHARED_CONTRACT.md, SHARED_SYMBOL_REGISTRY.md, MODULE_DEPENDENCY_MAP.md, DATA_MODEL_AND_INTERFACES.md, CODING_CONVENTIONS.md, ERROR_HANDLING_AND_SECURITY.md, IMPLEMENTATION_PHASE_PLAN.md, CHANGE_CONTROL.md, GLOBAL_ACCEPTANCE_TESTS.md, PROGRESS.md and a project-specific OpenCode master prompt plus phase prompts.

20. Error, offline and edge-case behavior

Situation	Expected behavior
Speech recognition unavailable/noisy	Show what was recognized if available; offer retry and typing. Never discard the intended task without notice.
AI API unavailable/rate limited	Use retry/backoff where suitable and a local/manual entry fallback; core saved tasks remain available.
AI output invalid or incomplete	Validate the structured result; reject/repair safely and ask only for essential clarification.
Not enough time	Show infeasibility and options; do not invent empty time or silently drop tasks.
No feasible candidate schedule	Preserve current schedule until an alternative is confirmed or apply an explicitly configured safe policy; explain the unresolved conflict.
Overlay permission denied	Keep core app functional; use permitted in-app/widget/notification alternatives.
Notification permission denied	Do not repeatedly ask; show tasks in app/widget and let user revisit settings.
User dismisses card	Close card without changing task status; do not immediately show the same card again.
Timezone/daylight-saving/time changed	Recalculate affected future schedule safely, preserve absolute event semantics and show important changes.
Device reboots/app process dies	Persist tasks and accepted schedule; reschedule supported work using Android-compliant mechanisms; do not duplicate alerts.
Database migration fails	Avoid data loss; provide a recoverable error path and tests for migrations.
Calendar sync conflict	Do not silently overwrite external changes; surface conflict and preserve source ownership/last sync metadata.
Undo after new changes	Detect stale/conflicting undo; avoid restoring an old schedule over later accepted user changes.

21. Implementation roadmap

The architecture and phase count are Claude's responsibility, following the Universal OpenCode prompt. The sequence below describes the product capabilities that should emerge. Claude may split/merge technical phases if justified, while keeping requirements traceable and work balanced.

Stage	Outcome	Key deliverables / gate
0. Requirements clarification	Clarify only consequential unknowns before architecture.	Claude summarizes project, lists knowns/unknowns and proposed defaults, asks a short grouped set of questions, then waits.
1. Foundation and design system	Runnable Android project from both Ubuntu and Windows.	Pinned toolchain, navigation, theme, accessible components, repository docs, app skeleton, basic CI/build checks.
2. Tasks and local data	Tasks can be added, viewed, edited and completed.	Task model, persistence, Today/Schedule basics, validation, task states, database tests.
3. Voice/text intent	Natural voice/text creates correct structured tasks.	Tap-to-speak, typed fallback, parser adapter, structured validation, correction flow, parser tests.
4. Scheduling and recovery	Produce and validate realistic schedules.	Constraints, deadline/priority model, fixed commitments, overload warnings, schedule revision/undo, scenario tests.
5. Board and relationships	Paper notes and red threads represent tasks/events.	Board interactions, relation types, filters, accessibility alternative, link integrity tests.
6. Quest Cards and reminder manager	Useful low-interruption reminders.	Optional overlay permission flow, compact panel, actions, reminder cap/quiet hours, notification fallback, Android-version/device testing.
7. Privacy, resilience and integration hardening	Reliable research-quality prototype.	Privacy centre, permission/revocation behavior, error cases, security review, offline handling, migration and cross-OS checks.
8. Evaluation and packaging	Ready-to-demonstrate URE submission.	Acceptance suite, study protocol/consent, metrics, demo scenario, README, architecture pack, limitations report, APK/build instructions.

Advanced personalization, cloud sync, multiple calendar providers, task suggestion automation and expanded gamification should be added only after the core loop is stable or if a validated URE need makes them necessary. Keep the architecture extensible without building every future feature prematurely.

22. Testing and verification strategy

22.1 Testing must be continuous

Each OpenCode phase must implement its actual files, run the relevant tests, report exact commands/results, update PROGRESS.md and the change log, and stop at clear acceptance gates. Do not defer integration until the final stage or claim unrun tests passed.

22.2 Test layers

Test layer	What to verify	Examples
Unit tests	Small pure logic works as specified.	Priority ranking, slot placement, deadline checks, duration estimates, relationship semantics, card caps.
Parser/contract tests	AI/voice output becomes valid canonical data.	Missing field, bad date, ambiguous intent,

Test layer	What to verify	Examples
		malformed JSON, prompt injection text, low confidence.
Schedule engine tests	Constraints and recovery rules hold for many cases.	Fixed events unchanged, no overlaps, deadline risk visible, task not silently dropped, split minimum respected.
Persistence tests	Tasks and revisions survive restarts and schema migration.	Create/update/delete, relation integrity, migration from prior schema, undo revision consistency.
Integration tests	Modules connect through canonical contracts.	Voice intent -> task -> schedule -> card -> Done -> history.
Android UI tests	Navigation, compact interactions and permissions behave.	Task card close/done/later, overlay permission denied, notification permission denied, large font.
Device tests	Android versions and manufacturers behave realistically.	Overlay, notification delivery, reboot, battery restrictions, microphone capture, gesture/navigation bars.
Cross-host build checks	Same repository can be built and tested from Ubuntu and Windows.	Clean clone, wrapper build, unit tests, lint/format, package build.
Security/privacy checks	Permissions and records are constrained.	No secrets in APK/source, unauthorized user access denied, revoke integration, delete/export, no unexpected recording.
Usability/research tests	People understand the app and are not overwhelmed.	First task time, task completion, reminder acceptance/dismissal, perceived realism and interruption burden.

22.3 Mandatory scenario tests

ID	Scenario	Pass criteria
AT-01	Late start: a user starts two hours late with several tasks and fixed commitments.	Fixed commitments remain, flexible tasks are sensibly moved, conflicts are detected, explanation is concise.
AT-02	Overloaded day: required work exceeds time.	Chrona labels the plan infeasible, prioritizes transparently, offers alternatives and never silently drops work.
AT-03	Repeated postponement of one task.	Chrona can offer a shorter session, split or alternative time without shaming or creating repeated reminders.
AT-04	Task is completed from Quest Card.	Task status updates, schedule recalculates if needed and the task disappears from next-action queue.
AT-05	Card is dismissed.	Task remains open; no immediate duplicate card; history records dismissal only if required by privacy settings.
AT-06	User denies overlay permission.	Core app remains usable; overlay stays off; fallback works without repeated permission nags.
AT-07	User denies notification permission.	App explains the limitation once and remains useful via in-app/available widget surfaces.
AT-08	Project is linked to a future showcase event.	Thread persists across dates; relationship reason is visible; relation does not invent a hard dependency.
AT-09	A true dependency is added.	Dependent task is scheduled after prerequisite

ID	Scenario	Pass criteria
		where feasible, or conflict is clearly reported.
AT-10	Speech contains ambiguous date/task.	Chrona asks at most a concise essential question or displays an editable interpretation; no false commit.
AT-11	AI/network unavailable.	User can create/edit stored tasks manually; no task loss; retry/fallback is understandable.
AT-12	Switch development host from Ubuntu to Windows or vice versa.	Clean checkout builds with documented tooling; no absolute paths or missing local state are required.
AT-13	Device time zone changes.	Future events and schedule blocks remain logically correct or changes are explained; no silent duplicate tasks.
AT-14	User deletes account/data or disconnects an integration.	Documented scope is deleted/disconnected; tokens cease to be usable; result is confirmed truthfully.
AT-15	Board has many notes and threads.	Pan/zoom/filter remains usable, text is readable and graph rendering remains responsive.

22.4 Test data and reproducibility

Create deterministic fixtures with tasks, deadlines, fixed events, dependencies, support links and expected behavior. Record random seeds if randomness is used. Keep tests independent of live AI/network calls by using a mock parser for scheduler tests. Use a small set of manual real-device tests for microphone, overlay, notification and OEM-specific behavior, because these cannot be proven fully by unit tests alone.

22.5 Build/test commands and reports

Claude/OpenCode must select actual commands after choosing the stack and put them in README and TESTING_AND_VERIFICATION_GUIDE.md. At minimum report clean build, static checks, unit tests, integration/UI tests where available, APK generation, and tests that could not be executed due to missing device/credential/external service. Never mark a test passed unless it actually ran successfully.

23. How we know Chrona is realistic and helpful

23.1 Schedule-quality metrics

Metric	Definition / purpose
Hard-constraint violation rate	Count of overlapping tasks, fixed-event movement without approval, or dependency-order violations. Target: zero in accepted schedules.
Deadline-risk visibility	Share of tasks that may miss deadlines that are correctly flagged before the user relies on the plan.
Silent task loss	Number of open tasks omitted or deleted without an explicit user decision. Target: zero.
Schedule correction rate	How often a user must manually correct a proposed schedule. Lower is better, provided task completion and preference fit do not worsen.
Estimation error	Difference between estimated and actual duration where actuals are available and consented.

Metric	Definition / purpose
Recovery time	Time/clicks/edits needed to get a usable plan after a defined disruption compared with a baseline approach.
Plan feasibility judgment	User rating of whether the revised plan felt achievable; compare against algorithmic checks.
Task outcomes	Important tasks completed, deferred or missed; interpret carefully and avoid attributing causation from small samples.

23.2 Reminder-quality metrics

Metric	Definition / purpose
Interruptions per user/day	Count of Quest Cards and notifications actually surfaced; separate urgent and non-urgent.
Dismiss-without-action rate	How often the card is dismissed without task action; not automatically "bad," but useful for tuning.
Useful-action rate	Share of cards that lead to Start, Done, Later or viewing meaningful task details.
Repeated-reminder rate	How often the same task is shown too often under the user's policy; target low and audit edge cases.
Permission opt-in / disable rate	Understand if users trust or dislike overlays/notifications; never treat opt-out as failure of user.
Perceived interruption burden	Short user rating after use; measure whether Chrona feels less annoying than their usual reminders.

23.3 Research discipline

For a small URE pilot, report sample size, participant profile at an appropriate aggregate level, scenarios, duration, consent, method, missing data, limitations and actual results. Avoid overstating statistical significance or claiming the app "improves productivity" from a small uncontrolled demo. Compare against a baseline such as manual schedule repair or a basic reminder list.

24. Release scope: demonstrator, research-ready and production path

Capability	URE demonstrator / first complete loop	Research-ready follow-up	Production path
Task capture	Voice and typing, core parse and edit.	Measure parse corrections; improve ambiguity handling.	More robust languages/accessibility and provider fallback.
Schedule	Deterministic feasible scheduling with priority/deadline/fixed events.	Tune with user scenarios and actual duration data.	Advanced constraints and reliable sync across calendars.
Recovery	Late-start, delay, missed task, overload; explanation and undo.	Compare recovery quality with baseline and user preference.	More recovery options and robust history/versioning.
Quest Cards	Optional overlay, compact panel, Done/Later/Dismiss; reminder cap and fallback.	Experiment with timing/caps; evaluate interruption burden.	Device/OEM compatibility and polished permission education.

Capability	URE demonstrator / first complete loop	Research-ready follow-up	Production path
Board/thread	Paper note look, priority colors, user-managed relation types.	Usability test with projects and cross-date links.	Larger boards, search, advanced filtering and sync.
Learning	Basic optional interaction/history data.	Personal duration estimates and reminder preference adaptation.	Privacy controls and explainable/resettable personalization.
Integrations	Not required; manual entry must work.	One opt-in calendar integration if justified.	Additional official APIs after privacy/security review.
Security/privacy	Local data and minimal external processing; clear permission handling.	Review data flow and consented metrics.	Hardened account/sync, backups, deletion and incident response.
Research	Repeatable scenarios and demonstration.	Pilot study, comparison and analysis.	Ongoing evaluation and app quality process.

Scope rule

Aim for production-quality engineering practices, not a production-sized feature backlog. The first release should form one complete end-to-end loop; features that do not strengthen task capture, realistic planning, recovery, quiet guidance, task relationships or verifiability should be deferred.

25. Differentiation, competitor claims and IP caution

The product must not claim novelty solely because it uses voice, overlays, colored notes, gamification or automatic rescheduling. Those interaction patterns and scheduling concepts exist in other products. The potential differentiator is the integrated behavior: task understanding with low-friction input; an explicit, realistic recovery engine; explained changes and preserved user control; a relationship graph shown as an approachable paper-and-thread board; and a notification decision system that deliberately limits interruptions.

Claude should treat this as a product positioning hypothesis and, if competitive claims are needed, conduct current competitor research and cite official, date-stamped evidence. Do not silently imply that the feature set has been proven unique.

Potential paper/patent direction should be considered only after an evidence-backed technical contribution is clearer. Keep dated design records, architecture changes, test data, experiment protocol and results. Avoid public claims of patentability. Patentability requires a qualified search and legal assessment of novelty, inventive step/non-obviousness, eligibility and jurisdiction-specific rules; publication and patent outcomes are not guaranteed.

26. Open decisions, assumptions and risk register

26.1 Settled product direction

- Working name: Chrona (provisional).
- Initial product target: Android application.
- Development must be portable between Ubuntu and Windows machines using one Git repository.
- Voice-first, typed fallback; explicit tap-to-speak is the default. No always-listening behavior in the initial scope.
- Core purpose: create a realistic plan and repair it when life changes.
- Quest Card: temporary, game-inspired task reminder with compact detail panel, not a full-app launch for every interaction.
- Quiet reminders: prevent notification overload; allow reminder caps, quiet hours, batching and user control.

- Paper-and-thread visual language: priority-colored notes, pins, red strings with meaningful task/event relationships.
- Privacy-first direction: no forced integrations; data access must be explicit and limited.
- Claude plans; one OpenCode agent builds sequentially in the same repository using canonical contracts and test gates.

26.2 Proposed defaults that can be revised

Topic	Proposed default	Reason / validation
UI/stack	Native Android; Kotlin + Jetpack Compose as first candidate.	Best aligned to native overlay, permission and Android UX requirements; verify tooling versions/feasibility.
Storage	Local-first MVP.	Reduce setup and privacy exposure; revisit if multi-device sync is required.
AI role	AI extracts structured intent; deterministic engine schedules.	Testability and hard-constraint safety.
Initial integrations	None required; consider read-only calendar later.	Avoid long setup and broad permissions.
Reminder default	Quiet mode with a small non-urgent cap; test two cards/day as a hypothesis.	Reduce interruption burden; validate with users.
Relationship suggestions	User-created first; suggested links require confirmation.	Prevent false inferred dependencies.
Learning	Optional, gradual, resettable.	Trust and privacy; defer complex learning until core loop is testable.

26.3 Questions Claude should ask only if needed

Following the Universal OpenCode prompt, Claude must first summarize the requirements and ask a compact set of consequential questions, then stop. It must not interrogate the user about ordinary implementation details. The following questions are the only likely architecture/product questions worth surfacing if the supplied documents do not resolve them:

9. Is the MVP allowed to use an external AI service for speech/text interpretation, or must task understanding be fully on-device? Recommended default: external AI may be used only behind a clear privacy disclosure and replaceable provider interface; core task storage and scheduling remain functional without it.
10. Is local-only storage acceptable for the URE demo, or is cloud sync/multiple-device access required from the first build? Recommended default: local-first, no account required, with sync deferred.
11. Which Android version/device should be the minimum real-device test target? Recommended default: Claude selects a currently supported baseline after checking the app's needed overlay/notification behavior and the developer's test device.
12. Should the overlay be required for the first demonstrator or be an optional capability with a notification/in-app fallback? Recommended default: optional overlay plus fallback because Android permissions and device behavior vary.
13. Is calendar integration mandatory for URE or an optional extension? Recommended default: not mandatory; model fixed commitments manually first, then add a narrow opt-in calendar connection if time allows.
14. Are there formal URE submission constraints (deadline, required research method, app demo requirements, report format) that should alter the implementation plan? If unknown, proceed with a normal milestone-based plan and mark them as external deliverables to confirm.

Claude should not ask questions already answered in this document or in the Universal OpenCode prompt. If these defaults are acceptable and no critical ambiguity remains, Claude should ask the user to confirm proceeding and wait, exactly as the universal prompt requires.

26.4 Main risks and mitigations

Risk	Impact	Mitigation
Overlay constraints vary across Android versions/OEMs	Quest Cards may not appear in every context.	Optional overlay, fallback, permission explanation, real-device testing; do not promise universal display.
AI misunderstands task or time	Incorrect plan or loss of trust.	Structured schemas, confidence, user correction, deterministic validator and test fixtures.
Scheduler optimizes for packing but not realism	User rejects plans or abandons app.	Buffers, user preferences, duration learning, overload state and measured correction rates.
Too many reminders	User disables Chrona.	Caps, quiet hours, batching, repeated-dismiss suppression and interruption metrics.
Board becomes visually cluttered	Relationships become unreadable.	Filters, project focus, zoom, list view, search and clear relation types.
Privacy claims exceed actual safeguards	Loss of trust and possible compliance issues.	Data inventory, permission minimization, provider disclosure, security checks and truthful policy.
Cross-OS build drift	Cannot switch computers reliably.	Pinned wrappers/toolchains, Git, clean-clone checks on both OSes, no local absolute paths.
Scope too large for two-credit URE	Incomplete or untestable demo.	One end-to-end MVP loop; defer advanced integrations/learning; explicit phase gates.
Novelty claimed before research	Weak paper/patent argument.	Competitor review, measured evaluation, documented technical contribution, qualified IP review.

Appendix A. Requirement ID families

Family	Coverage
PRJ / UX	Product purpose, onboarding, navigation and visual design.
VOICE / PARSE	Speech, typed capture, intent parsing, clarification and correction.
TASK / DATA	Task/event fields, states, persistence and relationships.
SCHED / RECOVERY	Feasible scheduling, constraints, overload handling, replanning and undo.
CARD / NOTIFY	Quest Cards, notification caps, quiet hours, permission/fallback.
BOARD / REL	Paper notes, red threads, relation types, filtering and accessibility.
LEARN	Optional personalization and behavior history.
PRIV / SEC	Data minimization, permissions, AI provider disclosures, secrets and access control.
PLAT / DEV	Android compatibility and Ubuntu/Windows development parity.
NFR	Quality, performance, reliability, maintainability and accessibility.
TEST / RES	Unit/integration/UI/device tests, metrics and research evaluation.

Claude should create a detailed REQUIREMENT_TRACEABILITY.md that assigns a stable ID to every meaningful requirement, with source, behavior, priority, owner module, implementation phase, files, shared contract IDs, verification method and status. IDs in this document are illustrative families; Claude may normalize the complete registry as long as traceability is preserved.

Appendix B. Glossary

Term	Meaning in Chrona
Task	A piece of work the user intends to do.
Event/commitment	A time-bound meeting, class, appointment or occurrence.
Fixed commitment	A block that the scheduler must not move without explicit user action.
Flexible task	A task that may move within constraints.
Dependency	A relation where one task must precede or unlock another.
Supportive relationship	A relation where one task helps another but does not necessarily block it.
Schedule repair/recovery	Creating a revised feasible plan after time/priority/availability changes.
Quest Card	A small, temporary Chrona task reminder shown in-app or as an optional overlay.
Overlay permission	Android permission enabling a window to appear above other apps in permitted circumstances.
Quiet hours	User-defined times when nonessential Chrona interruptions should be suppressed.
Local-first	Core data/functions remain usable locally without requiring constant network access.
Canonical contract	The authoritative definitions for shared types, functions, schemas and cross-module interfaces.
OpenCode phase prompt	A focused instruction for one implementation phase, used by the same OpenCode agent in the same repo.

Appendix C. Authoritative technical references

These references ground the Android/platform notes in this document. They are not a substitute for checking the current official documentation when the architecture and target SDK are selected. Accessed for this draft on 14 October 2026.

Topic	Official reference
Android notification runtime permission	https://developer.android.com/develop/ui/compose/notifications/notification-permission
Android notification channels	https://developer.android.com/develop/ui/compose/notifications/channels
Background foreground-service start restrictions	https://developer.android.com/develop/background-work/services/fgs/restrictions-bg-start
Foreground-service type and microphone requirements	https://developer.android.com/develop/background-work/services/fgs/service-types
Google API OAuth scopes / permissions	https://developers.google.com/identity/protocols/oauth2/scopes

The supplied Universal Software Development Master Prompt (19-page PDF) is also an authoritative process reference for Claude's planning obligations, shared contracts, single-agent/single-workspace implementation workflow, phase prompts, progress tracking, testing, change control and final consistency audit.

Appendix D. Instructions for Claude

Treat this as the product requirements baseline

Read this document together with Universal_OpenCode_Project_Development_Prompt.pdf. During Phase 0, summarize the project and constraints in simple terms, identify explicit requirements vs proposed defaults vs unknowns, ask only questions that materially affect scope, platform, architecture, privacy, cost or essential behavior, and STOP for the user's answers. After clarification, create the complete architecture and development pack described by the universal prompt. Do not implement source code during planning. OpenCode is the only coding agent and must implement sequentially in one repository, with shared contracts, tests, progress updates and honest verification reports.

Where this document leaves technical choices open, Claude should recommend the simplest secure, testable and cross-platform-development-compatible option, verify version-sensitive technical details against official sources, record the decision, and avoid introducing paid services or broad data access without approval.